

Eric Miller Project 2

First, I import all the libraries I'll need for the project

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

Then I read in the datasets needed for the project. I renamed some columns. I could not find the HDI info in the WDI dataset, so I had to use another dataset and then merge them. I then had to clean and prep the data for merging

```
hdi_raw = pd.read_csv("hdi-data.csv")

hdi = (
    hdi_raw.loc[hdi_raw["indicatorCode"] == "hdi", ["countryIsoCode", "country", "year", "value"]]
    .rename(columns={
        "countryIsoCode": "iso",
        "country": "country_name",
        "value": "hdi"
    })
)
```

I also had to convert the columns into numerical data types so it could be useful for modeling

```
wdi_raw = pd.read_csv("wdi-data.csv")

year_cols = [c for c in wdi_raw.columns if "VR" in c]

wdi_long = wdi_raw.melt(
    id_vars=["Country Name", "Country Code", "Series Name", "Series Code"],
    value_vars=year_cols,
    var_name="year",
    value_name="value"
)
wdi_long["year"] = wdi_long["year"].str.extract(r"(\d{4})").astype(int)
```

I then filter down the dataset to just include the variables I need and restructure it so it can be used for modeling

```
INDICATOR_MAP = {
    "life_expectancy": "SP.DYN.LE00.IN",
    "gdp_pc_current_usd": "NY.GDP.PCAP.CD",
    "internet_users_pct": "IT.NET.USER.ZS",
    "secondary_enrollment_pct": "SE.SEC.ENRR",
    "female_labor_share_pct": "SL.TLF.TOTL.FE.ZS",
    "co2_tons_per_capita": "EN.GHG.CO2.PC.CE.AR5",
    "health_exp_pc_usd": "SH.XPD.CHEX.PC.CD"
}

wdi_long = (
    wdi_long[wdi_long["Series Code"].isin(INDICATOR_MAP.values())]
    .drop(columns=["Series Name"])
)

wdi_wide = (
    wdi_long
    .pivot(
        index=["Country Code", "Country Name", "year"],
        columns="Series Code",
        values="value"
    )
    .reset_index()
)
wdi_wide.columns.name = None
```

I then made the columns more readable and then merged the two datasets. I Only kept rows that matched both country and year. This made it where I now have all relevant information on one row

```
reverse_map = {v: k for k, v in INDICATOR_MAP.items()}

rename_map = {
    "Country Code": "iso",
    "Country Name": "country_name"
}
rename_map.update(reverse_map)

wdi_wide = wdi_wide.rename(columns=rename_map)
```

```
merged = pd.merge(
    hdi,
    wdi_wide,
    on=["iso", "year"],
    how="inner"
)

merged = merged.drop(columns=["iso"])
```

I then define what columns need to be numeric and convert them, also replacing errors with NaN, then dropping all NaNs. Then I preview the dataset to check everything is correct

```
numeric_cols = [
    "hdi",
    "gdp_pc_current_usd",
    "life_expectancy",
    "secondary_enrollment_pct",
    "internet_users_pct",
    "female_labor_share_pct",
    "co2_tons_per_capita",
    "health_exp_pc_usd"
]

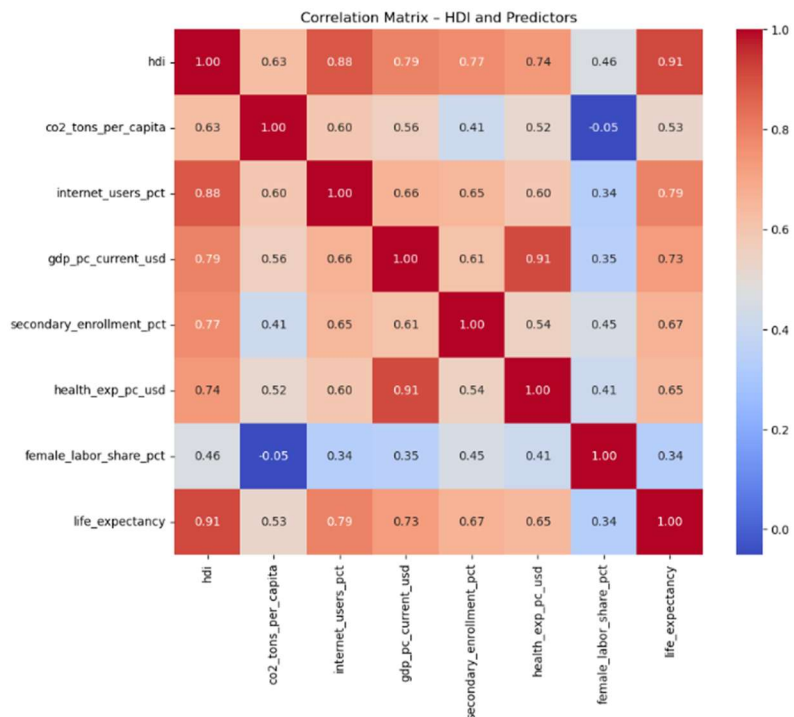
for col in numeric_cols:
    merged[col] = pd.to_numeric(merged[col], errors="coerce")

model_data = merged.dropna(subset=numeric_cols)

model_data.head()
```

	country_name_x	year	hdi	country_name_y	co2_tons_per_capita	internet_users_pct	gdp_pc_current_usd	secondary_enrollment_pct	health_exp_pc_usd
0	Afghanistan	2014	0.497	Afghanistan	0.238643	7.00	625.054942	54.293652	60.049
1	Afghanistan	2015	0.496	Afghanistan	0.246706	8.26	565.569730	53.150108	59.919
2	Afghanistan	2016	0.495	Afghanistan	0.216930	11.00	522.082216	53.435040	61.372
3	Afghanistan	2017	0.496	Afghanistan	0.226011	13.50	525.469771	55.536430	66.823
4	Afghanistan	2018	0.498	Afghanistan	0.215878	16.80	491.337221	57.357849	71.225

I then make a correlation matrix to see how all our predictors correlate. We can see most are strongly correlated with HDI



```
plt.figure(figsize=(10, 8))
corr = model_data.drop(columns=["country_name_y", "country_name_x", "year"]).corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix - HDI and Predictors")
plt.show()
```

I then made a scatter plot for each predictor against HDI

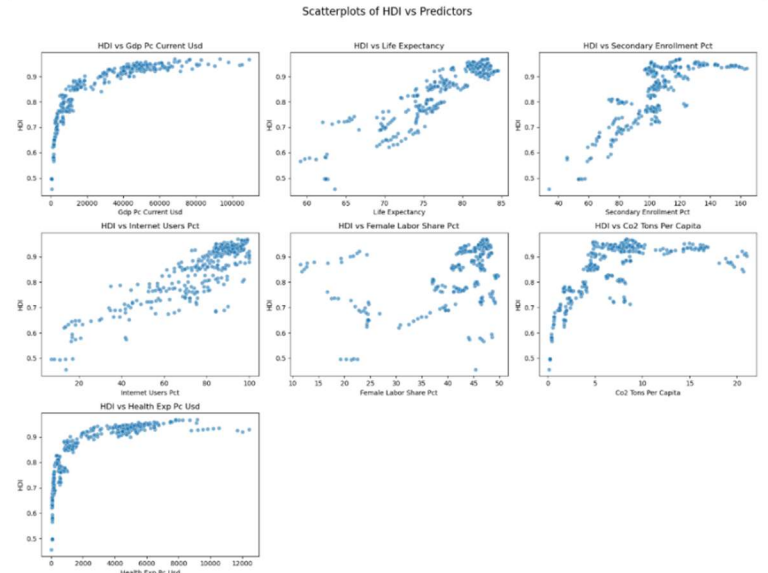
```
predictors = [
    "gdp_pc_current_usd",
    "life_expectancy",
    "secondary_enrollment_pct",
    "internet_users_pct",
    "female_labor_share_pct",
    "co2_tons_per_capita",
    "health_exp_pc_usd"
]

num_vars = len(predictors)
cols = 3
rows = (num_vars // cols) + int(num_vars % cols > 0)

plt.figure(figsize=(16, 12))

for i, var in enumerate(predictors):
    plt.subplot(rows, cols, i + 1)
    sns.scatterplot(
        x=var,
        y="hdi",
        data=model_data,
        alpha=0.6
    )
    plt.title(f"HDI vs {var.replace('_', ' ').title()}")
    plt.xlabel(var.replace('_', ' ').title())
    plt.ylabel("HDI")

plt.suptitle("Scatterplots of HDI vs Predictors", fontsize=16, y=1.02)
plt.tight_layout()
plt.show()
```



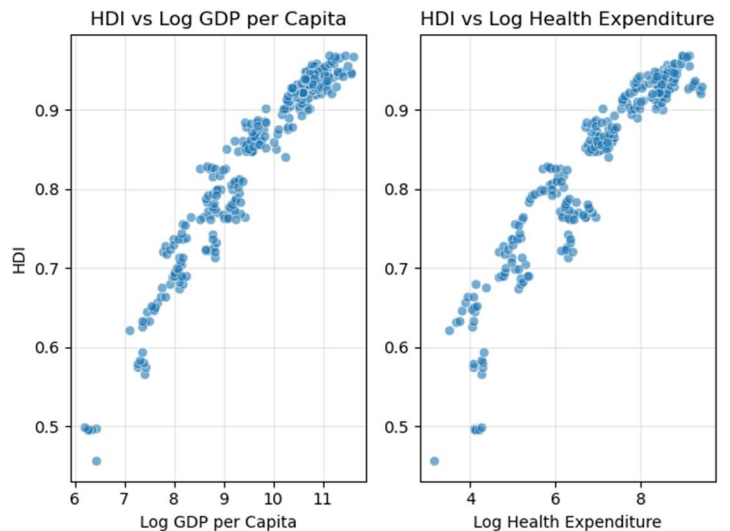
Looking at the scatter plots I saw both GDP per capita and Health expenditure would be helped out majorly with a log transformation. The values were very large and by logging them it made them more proportional and better for modeling

```
model_data["log_gdp_pc"] = np.log(model_data["gdp_pc_current_usd"])
model_data["log_health_exp_pc_usd"] = np.log(model_data["health_exp_pc_usd"])

plt.subplot(1, 2, 1)
sns.scatterplot(
    x="log_gdp_pc",
    y="hdi",
    data=model_data,
    alpha=0.6
)
plt.title("HDI vs Log GDP per Capita")
plt.xlabel("Log GDP per Capita")
plt.ylabel("HDI")
plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
sns.scatterplot(
    x="log_health_exp_pc_usd",
    y="hdi",
    data=model_data,
    alpha=0.6
)
plt.title("HDI vs Log Health Expenditure")
plt.xlabel("Log Health Expenditure")
plt.ylabel("")
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



We then ran a model using all our predictors, with the log transformed GDP and health expenditures

```

predictors_log = [
    "log_gdp_pc",
    "life_expectancy",
    "secondary_enrollment_pct",
    "internet_users_pct",
    "female_labor_share_pct",
    "co2_tons_per_capita",
    "log_health_exp_pc_usd"
]

y = model_data["hdi"]
X = model_data[predictors_log]
X = sm.add_constant(X)

ols_model = sm.OLS(y, X).fit()
ols_model.summary()

```

Dep. Variable:	hdi	R-squared:	0.970
Model:	OLS	Adj. R-squared:	0.969
Method:	Least Squares	F-statistic:	1499.
Date:	Fri, 05 Dec 2025	Prob (F-statistic):	5.39e-242
Time:	19:20:09	Log-Likelihood:	840.94
No. Observations:	331	AIC:	-1666.
Df Residuals:	323	BIC:	-1635.
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1374	0.024	-5.763	0.000	-0.184	-0.090
log_gdp_pc	0.0433	0.005	8.809	0.000	0.034	0.053
life_expectancy	0.0054	0.000	12.813	0.000	0.005	0.006
secondary_enrollment_pct	0.0004	7.6e-05	5.231	0.000	0.000	0.001
internet_users_pct	0.0008	9.6e-05	8.500	0.000	0.001	0.001
female_labor_share_pct	0.0012	0.000	7.192	0.000	0.001	0.001
co2_tons_per_capita	0.0014	0.000	3.924	0.000	0.001	0.002
log_health_exp_pc_usd	-0.0023	0.003	-0.693	0.489	-0.009	0.004

Omnibus:	18.940	Durbin-Watson:	0.452
Prob(Omnibus):	0.000	Jarque-Bera (JB):	53.975
Skew:	0.088	Prob(JB):	1.90e-12
Kurtosis:	4.970	Cond. No.	3.59e+03

The R^2 was very high for the model, which is good, but one of our predictors, health expenditure per capita, had a very high p-value, meaning it was statistically insignificant, so I decided to remove it and run the model again

```

new_predictors_log = [
    "log_gdp_pc",
    "life_expectancy",
    "secondary_enrollment_pct",
    "internet_users_pct",
    "female_labor_share_pct",
    "co2_tons_per_capita"
]

y = model_data["hdi"]
X = model_data[new_predictors_log]
X = sm.add_constant(X)

ols_model = sm.OLS(y, X).fit()
ols_model.summary()

```

Dep. Variable:	hdi	R-squared:	0.970
Model:	OLS	Adj. R-squared:	0.970
Method:	Least Squares	F-statistic:	1751.
Date:	Fri, 05 Dec 2025	Prob (F-statistic):	1.57e-243
Time:	19:20:11	Log-Likelihood:	840.70
No. Observations:	331	AIC:	-1667.
Df Residuals:	324	BIC:	-1641.
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1285	0.020	-6.401	0.000	-0.168	-0.089
log_gdp_pc	0.0405	0.003	14.723	0.000	0.035	0.046
life_expectancy	0.0054	0.000	13.145	0.000	0.005	0.006
secondary_enrollment_pct	0.0004	7.56e-05	5.193	0.000	0.000	0.001
internet_users_pct	0.0008	9.56e-05	8.479	0.000	0.001	0.001
female_labor_share_pct	0.0011	0.000	7.196	0.000	0.001	0.001
co2_tons_per_capita	0.0014	0.000	3.972	0.000	0.001	0.002

I then wanted to use a train/test split to evaluate the model's predictive power

```
X_final = model_data[new_predictors_log]
y_final = model_data["hdi"]

X_train, X_test, y_train, y_test = train_test_split(
    X_full, y_full,
    test_size=0.2,
    random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_test = lr.predict(X_test)

r2 = r2_score(y_test, y_pred_test)
mae = mean_absolute_error(y_test, y_pred_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))

print("Test R-squared:", r2)
print("Test MAE:", mae)
print("Test RMSE:", rmse)
```

Test R-squared: 0.9456842413940455
 Test MAE: 0.019196863896977966
 Test RMSE: 0.02475713761636539

I plotted our predictions vs actual hdi data, and our predictions were very accurate

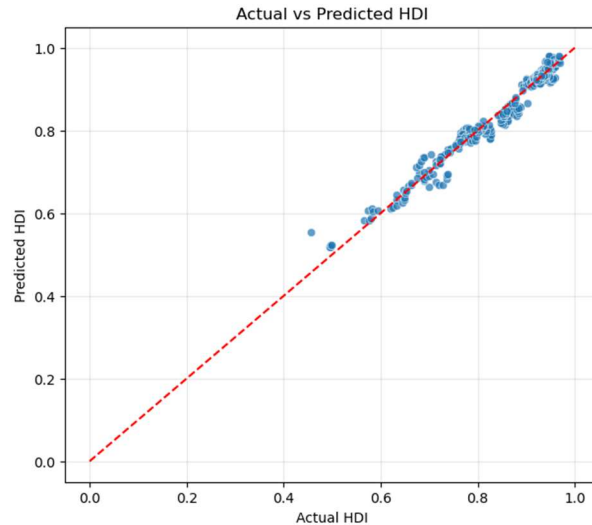
```
model_data["predicted_hdi_final"] = ols_model.fittedvalues

plt.figure(figsize=(7, 6))
sns.scatterplot(
    x="hdi",
    y="predicted_hdi_final",
    data=model_data,
    alpha=0.7
)

plt.plot([0, 1], [0, 1], "--", color="red", linewidth=1.5)

plt.xlabel("Actual HDI")
plt.ylabel("Predicted HDI")
plt.title("Actual vs Predicted HDI")
plt.grid(alpha=0.3)

plt.show()
```



I then wanted to know how our predictors would act over time, so I ran models for each year and made a table for each year with all the coefficients, R^2 and adjusted R^2

```
years = sorted(model_data["year"].unique())
results_by_year = []

for yr in years:
    df_year = model_data[model_data["year"] == yr].dropna()
    X = sm.add_constant(df_year[[
        "log_gdp_pc",
        "life_expectancy",
        "secondary_enrollment_pct",
        "internet_users_pct",
        "female_labor_share_pct",
        "co2_tons_per_capita"]])
    y = df_year["hdi"]
    model = sm.OLS(y, X).fit()

    results_by_year.append({
        "year": yr,
        "R2": model.rsquared,
        "Adj_R2": model.rsquared_adj,
        "coef_log_gdp": model.params["log_gdp_pc"],
        "coef_life_exp": model.params["life_expectancy"],
        "coef_education": model.params["secondary_enrollment_pct"],
        "coef_fem_labor": model.params["female_labor_share_pct"],
        "coef_co2_tons": model.params["co2_tons_per_capita"]
    })

results_df = pd.DataFrame(results_by_year)
results_df
```

	year	R2	Adj_R2	coef_log_gdp	coef_life_exp	coef_education	coef_fem_labor	coef_co2_tons
0	2014	0.977623	0.972459	0.016185	0.007139	0.000373	0.001643	0.003357
1	2015	0.972287	0.966744	0.037851	0.005276	0.000561	0.000468	0.000745
2	2016	0.981540	0.977848	0.033318	0.004961	0.000335	0.000684	0.000830
3	2017	0.971156	0.965388	0.040662	0.004715	0.000211	0.001426	0.001250
4	2018	0.970351	0.964613	0.045368	0.004171	0.000166	0.001296	0.000628
5	2019	0.961998	0.954398	0.049978	0.003466	0.000210	0.001161	0.000759
6	2020	0.970379	0.964454	0.047644	0.003832	0.000241	0.001123	0.000813
7	2021	0.968388	0.962065	0.041419	0.004867	0.000484	0.001047	0.001117
8	2022	0.972883	0.967634	0.036103	0.005866	0.000495	0.001561	0.001988

Although there are fluctuations in the coefficients over time, they mostly remain consistent in their influence